

# Tools : linter, formater, static checker

---

## 1. Pylint : linter for finding problems

pylint is a tool for finding bugs and style problems in Python source code. It finds problems that are typically caught by a compiler for less dynamic languages like C and C++. Because of the dynamic nature of Python, some warnings may be incorrect; however, spurious warnings should be fairly infrequent

```
pip install pylint
pylint --rcfile=./coding_guideline/pylintrc.google ./src
pylint --rcfile=./coding_guideline/pylintrc.google ./test
```

- Make sure you run pylint on your code.
- Check to understand and fix the warnings
- Many warnings can be auto fixed by using the fommater below, but there are warnings that we need to fix manually: for example I saw many time we used *type* instead of *isinstance*

## 2. Yapf : automatic formatting

```
pip install yapf
# format all files recursively in current folder
yapf -ir .
# format a file with google style (we can specify other style such as
# pep8, facebook, etc.)
yapf --style google file_path
```

For example in our devr-tensorflow project we can au-format the code by using these 2 command:

```
yapf --style google -ir ./src
yapf --style google -ir ./test
```

If we dont want yapf to format some part of code, just disable/enable yapf like this one below

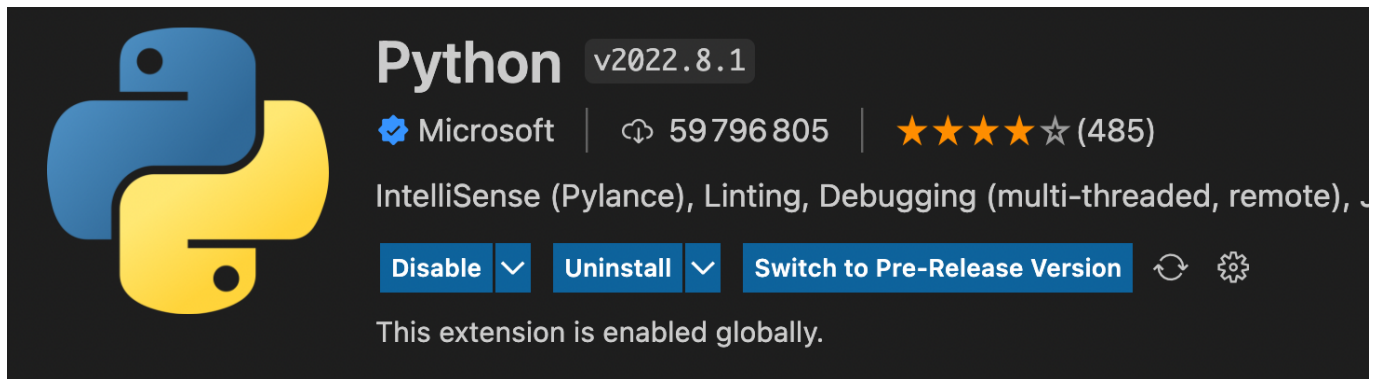
```
# yapf: disable
FOO = {
    # ... some very large, complex data literal.
}

BAR = [
    # ... another large data literal.
]
# yapf: enable
```

### 3. Setup plugins to use in VSC or IntelliJ

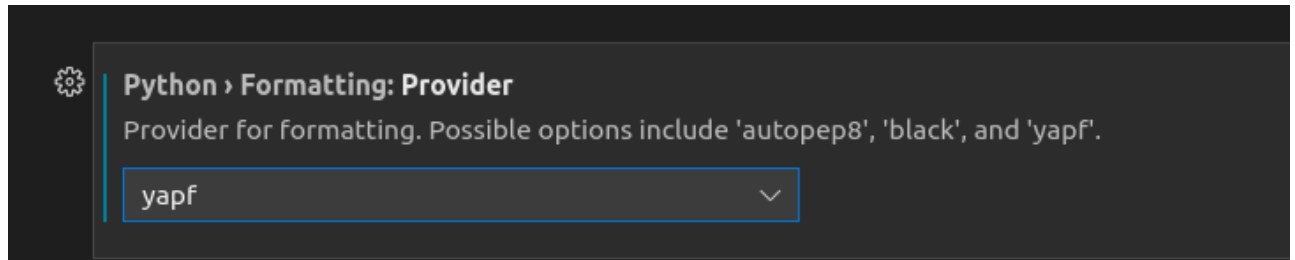
#### Linters and formater

We can install the "Python" extension in VSC or IntelliJ IDEA which support code checking (pylint, flake8, etc.) and code formatting (yapf, black, autopep8, etc.). For Pycharm, it is included in the PyCharm Pro version (or add specific tool like black, yapf, pylint with "External Tools" in PyCharm).



After installing this plugin, you will have directly all the warnings of code checking (pylint) when open a py file. You have also all the option to auto-format the code in right-click menu.

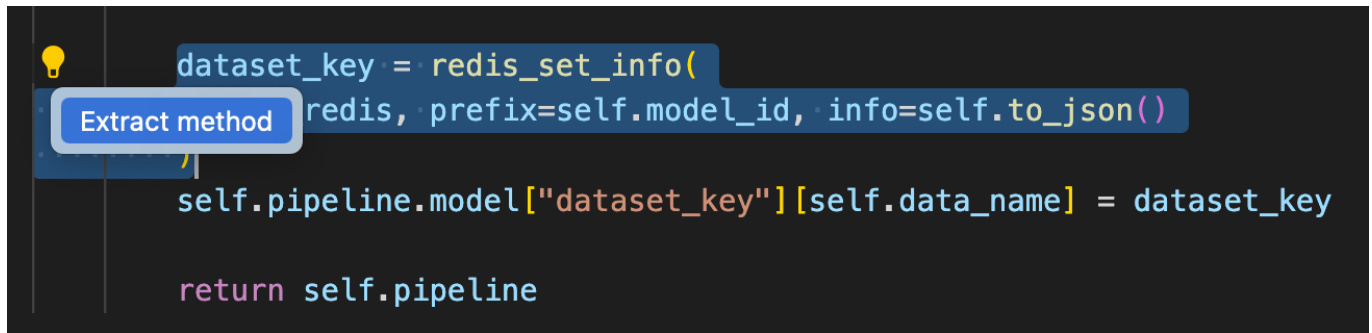
- How to config/use linter and select pylint: <https://code.visualstudio.com/docs/python/linting>
- How to config/use formater and select yapf: [https://code.visualstudio.com/docs/python/editing#\\_formatting](https://code.visualstudio.com/docs/python/editing#_formatting)



#### Refactoring feature of VSC Python plugin

- You can sort the imports follow the convention I mentioned in the coding\_guideline.md document.
- You can rename variable, module etc. in the project
- Extract method: you can select the lines of code you wish to extract as a method. Then select the light-bulb that is displayed next to it. **The plugin will replace all the code with a methode\_call every where.**
- Extract variable : the same principe as the extract method

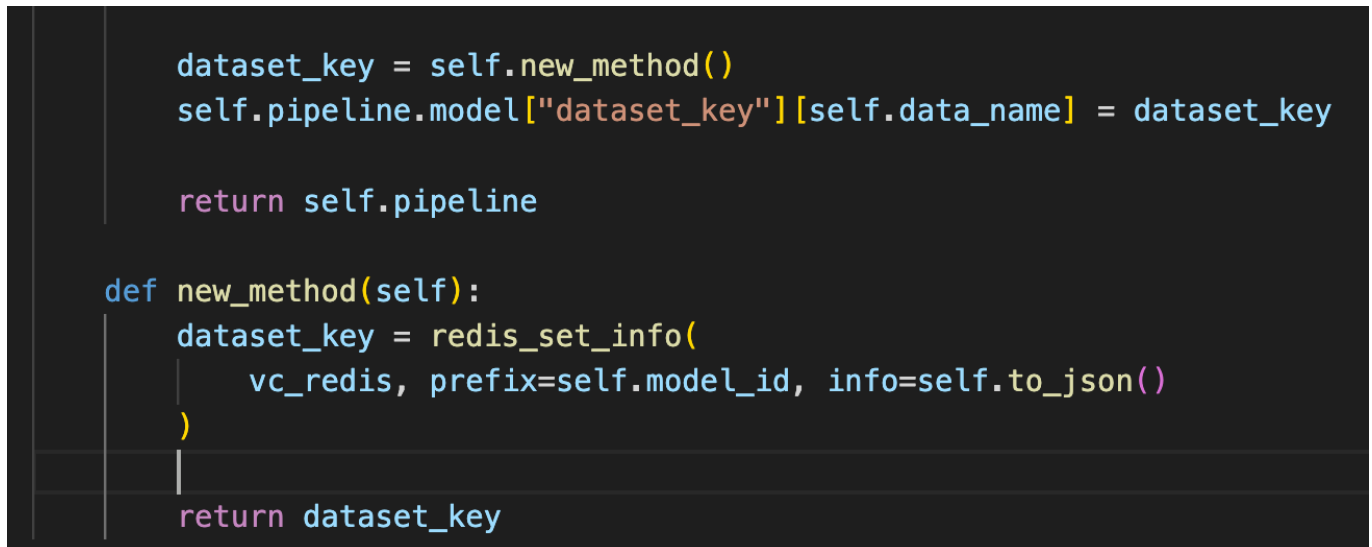
Before:



```
dataset_key = redis_set_info(
    redis, prefix=self.model_id, info=self.to_json()
)
self.pipeline.model["dataset_key"][self.data_name] = dataset_key

return self.pipeline
```

After:



```
dataset_key = self.new_method()
self.pipeline.model["dataset_key"][self.data_name] = dataset_key

return self.pipeline

def new_method(self):
    dataset_key = redis_set_info(
        vc_redis, prefix=self.model_id, info=self.to_json()
    )
    return dataset_key
```

## 4. Skip / Ignore warning in Pylint

Skip / ignore by error codes

- Look for the line `disable=<numerical_ID>` in the `[MESSAGES CONTROL]` section. Then add the code you want to ignore the warning.

[MESSAGES CONTROL]

```
# Enable the message, report, category or checker with the given id(s).
You can
# either give multiple identifier separated by comma (,) or put this
option
# multiple time.
#enable=

# Disable the message, report, category or checker with the given id(s).
You
# can either give multiple identifier separated by comma (,) or put this
option
# multiple time (only on the command line, not in the configuration file
where
# it should appear only once).
#disable=
```

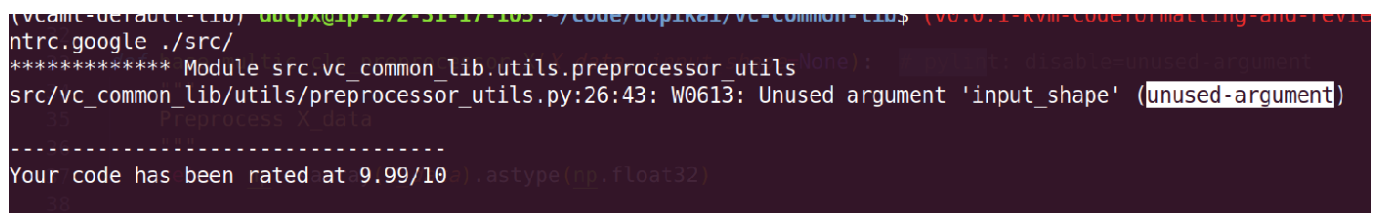
- For code C0103 (invalid-name), you can add names to disable in the `good-names` item of the `[BASIC]` section.

[BASIC]

```
# Good variable names which should always be accepted, separated by a
comma
good-names=
```

- We can edit other items, like `indent-string`, `exclude-protected`, etc.

Skip / ignore for a line or for a file or a block of code



```
(vcant-default-lib) ducpx@ip-172-31-17-105:~/Code/duprk1/vc-common-lib$ (v0.0.1-KvM-codeformatting-and-revie
ntrc.google ./src/
***** Module src.vc_common_lib.utils.preprocessor_utils (one):
src/vc_common_lib/utils/preprocessor_utils.py:26:43: W0613: Unused argument 'input_shape' (unused-argument)
-----
Your code has been rated at 9.99/10
```

- Locally disable a particular message: Add `# pylint: disable=<some-message>,<another-one>` at the desired block level or at the end of the desired line of code. The message is taken from the `( )` notation (end of the line). If there are multiple warnings in 1 line, add it after a comma.

Example:

```
# pylint: disable=unused-argument, multiple-statements, fixme, line-
too-long
```

- Disable a message for a particular module: Add the corresponding option in a comment at the top of the file. Example:

```
# pylint: disable=wildcard-import, method-hidden
# pylint: enable=too-many-lines
```

Quick key in VSC to auto correct issues

- Not available. See details here: [https://code.visualstudio.com/docs/python/linting#\\_pylint](https://code.visualstudio.com/docs/python/linting#_pylint)

## More complex refactoring

In this document we do not handle complexity refactoring. We will do it separately as it requires much more manual efforts.